```
mvn clean compile exec:java \
  -Dexec.mainClass=com.manning.apisecurityinaction.DtlsClient
```

> **TIP** If you want to see details of the DTLS protocol messages being sent between the client and server add the argument `-Djavax.net.debug=all` to the Maven command line. This will produce detailed logging of the handshake messages.

The client will start up, connect to the server, and send all of the lines of text from the input file to the server, which will receive them all and print them out. After the client has completed it will print out the DTLS cipher suite that it used. In the next section you'll see how the default choice made by Java might not be appropriate for IoT applications and how to choose a more suitable replacement.

> **NOTE** This example is intended to demonstrate the use of DTLS only and is not a production-ready network protocol. If you separate the client and server over a network, it is likely that some packets will get lost. Use a higher-level application protocol such as CoAP if your application requires reliable packet delivery (or use normal TLS over TCP).

12.1.2 Cipher suites for constrained devices

In previous chapters, you've followed the guidance from Mozilla[3] when choosing secure TLS cipher suites (recall